The Open University of Israel

Department of Mathematics and Computer Science

Identifying Likely Large-Scale Agile Implementation Pathways

ADVANCED PROJECT REPORT

Advanced Project in Computer Science (22997)

Advanced project submitted as partial fulfillment of the requirements
towards an M.Sc. degree in Computer Science at The Open University of Israel

Submitted by

Erez Morabia

Prepared under the supervision of

Prof. Shmuel Tyszberowicz

September 2026

Table of Contents

Abstract

This project addresses the critical challenge of large-scale agile transformation in organizations by developing a system that recommends likely next agile practices from organizational history. Its core innovation is empirically learning organizational improvement behavior to identify likely next practices: it derives team-specific guidance from observed peer-team maturity histories, observed practice-transition behavior, and organization-wide practice-improvement trends within the organization. Collaborative filtering, the Practice Transition Model, and time-aware popularity are blended under one policy selected automatically for each prediction month — never tuned on the month it predicts — across 87 teams, 35 practices, and 10 months of historical data. Walk-forward backtesting on the five prediction months with a complete outcome window shows this blend aligning with later improvements in 58.0% of evaluated cases, 2.2x the random baseline (26.0%), and 2.3 percentage points ahead of an equally walk-forward-selected time-aware-popularity comparison arm (55.7%) on the same cases; across all seven prediction months (including two with a truncated outcome window) the figures are 50.9% vs. 47.5%. This result is exploratory, not a claim of proven superiority over popularity alone (see §6.3, §6.5, and `src/ml/policy.py`). The system is a functional prototype — a working web interface and API, not a hardened production deployment — and is ready for pilot testing with selected teams, addressing the original proposal's objective of providing data-driven recommendations for agile adoption pathways. §7.3 details what would still be required to harden it for production use.

Executive Summary

The Challenge

Large organizations implementing agile transformation face a critical decision-making challenge: determining which agile practices each team should focus on next to maximize success probability. Manual analysis doesn't scale to the number of teams, practices, and maturity combinations involved, and there is no single authoritative source detailing the correct sequence of adoption steps. See §1.1 for the full problem statement, including the scale of data involved at the target organization (Avaya).

The Approach

The project's core innovation is not a new machine learning algorithm. It is the empirical approach of learning organizational improvement behavior from observed team histories, then using that evidence to identify likely next practices for an individual team. Collaborative filtering and the Practice Transition Model are the implementation tools that operationalize this approach:

1. Collaborative Filtering

- Analyzes organizational patterns from historical data (87 teams, 35 practices, 10 months)
- Finds teams similar to a target team based on their current practice maturity profiles
- Uses cosine similarity to measure team similarity
- Recommends practices that similar teams successfully improved

2. Practice Transition Model

- Learns empirical practice-to-practice transitions from consecutive improvement-bearing steps
- Identifies which practices typically follow others in organizational improvement patterns
- Ensures recommendations follow logical improvement pathways
- Prevents recommending practices teams aren't ready for

3. Time-Aware Popularity

- Tracks organization-wide practice-improvement frequency, restricted to practices a team hasn't already mastered
- Blends all-time popularity with the single most recent organization-wide transition

4. Global Monthly Adaptive Blend

- Blends similarity, sequence, and time-aware popularity evidence with weights selected once per prediction month, not per team and not fixed in advance
- The month-specific policy (peer count, similarity threshold, the three factor weights, and the popularity recency weight) is chosen from a fixed grid of 675 combinations by maximizing accuracy on strictly earlier prediction months whose outcomes have already closed
- Normalizes each component separately before combining

- Filters out practices already at maximum maturity
- Returns exactly two recommendations for an eligible team-month; when fewer than two non-maxed practices remain, it returns an empty list with an explanatory message

5. Validation Methodology

- Uses historical backtesting: for each prediction month, replay the policy that would have been selected at that point in time, then validate against actual improvements
- Rolling window approach: validates recommendations against actual improvements
- Accounts for adoption timelines (validates across a 3-snapshot window)
- Compares results against a random baseline and an independently-selected time-aware-popularity comparison arm, split into primary (complete outcome window) and sensitivity (all months) results

Successful Results

The system demonstrates strong performance and practical value:

Recommendation Accuracy (primary: five prediction months with a complete outcome window):

- **58.0% alignment** between recommended practices and later team improvements
- **2.2x improvement** over random baseline (26.0%)
- **2.3 percentage points ahead** of an independently, walk-forward-selected time-aware-popularity comparison arm on the same cases (55.7%)
- **121 evaluable cases** across multiple teams and months

How to interpret these results: All accuracy and improvement figures are aggregate backtest results across the organization (macro-averaged across tested months), and the comparison against time-aware popularity is exploratory, not a proven claim of superiority — three of the five primary months still fall back to a bootstrap policy (100% popularity) because no prior month had a completed outcome window yet, so the blend and the popularity arm tie exactly in those months. They describe how the model performed on historical validation cases and do not guarantee an improvement, recommendation match, or maturity outcome for any individual team or month. See §6.3 for the full per-month breakdown and the sensitivity results across all seven prediction months.

System Capabilities:

- Processes the project dataset efficiently (655 recorded team-month rows × 35 practices, or 22,925 team-practice cells before missing-data filtering)
- Working web interface for easy use by non-technical users (see §7.3 for what's still needed for full production deployment)
- Real-time recommendations based on current organizational data
- Global monthly policy selection replaces manual parameter tuning: each prediction month automatically re-selects its own blend from prior completed outcomes

Practical Readiness:

- System is ready for pilot deployment and testing with selected teams (not yet a hardened production deployment — see §7.3)
- Excel data format matches organizational data collection methods
- Comprehensive validation framework evaluates implementation results
- Can serve all 70+ teams simultaneously (vs. 1-2 teams manually)

Business Impact:

- Provides data-driven recommendations instead of intuition-based decisions
- Estimated to eliminate 4-8 hours/month of manual analysis per team (practitioner estimate, not a measured result — see §7.4)
- Standardizes approach across all teams for consistency
- Enables faster transformation by focusing teams on practices with highest success probability

The system successfully addresses the original proposal's objective of providing data-driven recommendations for agile adoption pathways, demonstrating that machine learning can effectively solve the large-scale agile transformation challenge.

---

1. Introduction

1.1 Problem Statement

Large organizations implementing agile transformation face a critical challenge: determining which agile practices each team should focus on next to maximize success probability. This problem is compounded by several factors:

- **Scale**: Organizations typically have 70+ teams, each at different maturity levels
- **Complexity**: 30+ different agile practices to choose from, each with multiple maturity levels (0-3)
- **Uniqueness**: Each organization's agile adoption process is unique due to differences in product characteristics, technology, organizational culture, and team sizes
- **Manual Analysis Impractical**: The volume of data (e.g., 70 teams × 30 practices × 4 maturity levels × multiple months/years) makes manual analysis impossible
- **No Authoritative Source**: There is no single written source detailing the correct sequence of steps for successful agile adoption

At Avaya (the target organization for this project), approximately 30 agile practices are tracked across 70 teams distributed across 10 time zones, with data collected and updated monthly. Analyzing all successful adoption pathways manually is impossible due to the vast amount of data and its frequent changes.

1.2 Project Objectives

The primary objective of this project is to build software capable of recommending to an agile team in an organization (at any given time) the next step to implement in the agile adoption process, such that this step has high success probability rates, based on the implementation history of other agile teams within the organization up to the current point in time.

Specific objectives include:

1. **Automated Recommendations**: Generate ranked lists of recommended practices for each team based on organizational patterns
2. **Empirical Organizational Learning**: Learn team-specific guidance from observed peer maturity histories and practice-transition behavior, using collaborative filtering and the Practice Transition Model as implementation tools
3. **Validation**: Validate recommendations against historical data using backtesting methodology
4. **Practical Deployment**: Create a system ready for real-world testing with selected teams

1.3 Scope and Limitations

Scope:

- Analysis of organizational data from 87 teams, 35 practices, 10 months

- Implementation of collaborative filtering and the Practice Transition Model
- Web-based interface for recommendations and validation
- Backtest validation methodology

Limitations:

- Recommendations are based on historical patterns and may not account for external factors
- A public web/API request must use a valid global prediction month (the fourth recorded global month or later), have a team snapshot in that prediction month and a baseline snapshot strictly before it, and retain at least two non-maxed practices
- Accuracy depends on data quality and completeness
- Recommendations are probabilistic, not deterministic guarantees

1.4 Connection to Original Proposal

This implementation directly addresses the original proposal's objectives:

- **Input Format**: Accepts Excel matrices with teams × practices × maturity levels (0-3), as specified in the proposal
- **Processing**: Receives team name as input and processes using machine learning algorithms
- **Output**: Provides exactly two ranked practices when at least two non-maxed candidates remain; callers cannot configure the count
- **Validation**: Uses historical backtesting methodology (train on past months, test on future months) as proposed
- **Practical Application**: System is ready for real-world testing with selected teams, as planned for May-July timeline

---

2. Background and Related Work

2.1 Large-Scale Agile Adoption Challenges

Agile adoption in large organizations has become increasingly common, with research showing that companies implementing agile processes have approximately 4x higher probability of success. However, organizations face significant challenges:

- **No Standard Path**: Agile implementations vary from organization to organization due to differences in product characteristics, technology, organizational culture, team sizes, and other factors
- **Long Duration**: Each organization's agile adoption process is unique and typically takes years to complete
- **No Authoritative Source**: There are dozens of recommended practices, but the order of implementation and intensity of rollout varies from organization to organization, with no single written source detailing the correct sequence
- **Resource Constraints**: A small number of agile coaches cannot manually analyze all pathways across a large organization's full team and practice volume, at the frequency (monthly) the data changes

(See §1.1 for the specific data volume and scale figures at the target organization, Avaya, that make manual analysis impractical.) Software capable of managing this volume of organizational data and identifying likely next adoption steps has significant business value to the organization.

2.2 Similarity-Based Recommendation

Similarity-based recommendation systems use collaborative filtering techniques to identify likely preferences by finding similar users or items. The underlying assumption is that users who agreed in the past tend to agree again in the future. This approach is particularly effective when dealing with large user-item matrices where explicit preferences are known.

Collaborative Filtering Concepts:

Collaborative filtering is a recommendation technique that identifies likely user preferences from the preferences of many users. The approach works by:

- **User-Item Matrix**: Representing users and items in a matrix where each cell contains a preference or rating value. In recommendation systems, this matrix captures user interactions with items.
- **Neighborhood-Based Approach**: Finding users (or items) similar to a target user and using their preferences to make recommendations. The assumption is that similar users will have similar preferences.
- **Memory-Based Methods**: Using the entire user-item matrix to compute recommendations, as opposed to model-based methods that learn a model from the data.

Similarity Metrics:

The effectiveness of collaborative filtering depends on accurately measuring similarity between users or items. Cosine similarity is a widely used metric that measures the cosine of the angle between two non-zero vectors in an inner product space.

Mathematical Formulation:

```
similarity(A, B) = (A · B) / (||A|| × ||B||)
```

Where:

- A and B are preference vectors (e.g., user ratings or item features)
- A · B is the dot product of the two vectors
- ||A|| and ||B|| are the magnitudes (L2 norms) of the vectors

Why Cosine Similarity:

Cosine similarity is particularly useful for recommendation systems because:

- **Direction Over Magnitude**: It measures similarity in direction rather than magnitude, making it robust to different rating scales or vector lengths
- **Normalization**: The cosine of the angle ranges from -1 to 1, providing a normalized similarity measure
- **Sparsity Handling**: Works well with sparse matrices common in recommendation systems where users rate only a subset of items
- **Computational Efficiency**: Can be computed efficiently using vector operations

Neighborhood-Based Recommendation:

Once similarity is computed, neighborhood-based collaborative filtering:

1. Identifies K most similar users (or items) to the target
2. Aggregates preferences from the neighborhood
3. Generates recommendations based on weighted preferences of similar users
4. Filters out items the user has already interacted with

This approach forms the theoretical foundation for similarity-based recommendation systems used in various domains, from e-commerce to content recommendation.

2.3 Practice Transition Model

The Practice Transition Model is an empirical summary of observed practice-to-practice transitions in the organization. For each team, it compares consecutive improvement-bearing steps and counts how often a practice improved in one step and another practice improved in the following step. Those counts are normalized into conditional transition probabilities used as a recommendation signal.

Key Concepts:

- **Transition Matrix**: Counts and conditional probabilities for observed practice-to-practice transitions
- **Improvement-Bearing Step**: A chronological step in which at least one practice improved; steps without improvements are skipped
- **Consecutive-Step Transition**: An ordered A → B relationship only when A occurs in one improvement-bearing step and B occurs in the next; practices that improve within the same step are not assigned an order

2.4 Hybrid Recommendation Approaches

Hybrid recommendation systems combine multiple recommendation techniques to improve accuracy and coverage. This project combines collaborative filtering (similarity-based) with sequence learning (content-based) to create a hybrid system that leverages both peer team patterns and organizational improvement sequences.

---

3. Methodology

3.1 System Architecture Overview

The system follows the input/processing/output architecture described in the original proposal:

Input:

- Excel matrices containing agile adoption data collected monthly
- The loader does not enforce a fixed minimum history. Recommendation months begin at the fourth recorded global snapshot; at least six global snapshots are needed for the first prediction month to have a complete 3-snapshot primary backtest outcome window
- Each matrix defined by:
  - **Y-axis**: List of teams in the organization
  - **X-axis**: List of agile practices
  - **Cell contents**: Maturity level score (0-3) indicating a team's maturity in a specific practice

Processing:

- Receives a team name as input
- Applies collaborative filtering to find similar teams
- Applies sequence learning to identify natural improvement patterns
- Combines signals using hybrid scoring

Output:

- Ranked list of agile practices recommended for the team to focus on as their next step
- Practices selected based on highest probability of success for the team's current adoption state
- Recommendations based on lessons learned from other teams' experiences within the same organization

3.2 Data Preprocessing

Data preprocessing involves several steps to prepare the Excel matrices for machine learning:

1. **Loading**: Read Excel file using pandas and openpyxl
2. **Validation**: Check for missing data, invalid values, and data quality issues
3. **Normalization**: Convert maturity scores from 0-3 scale to 0-1 range for consistent scaling
4. **Structure Building**: Build team histories indexed by month for efficient access

Normalization Formula:

```
normalized_score = raw_score / 3.0
```

This ensures all practice scores are in the [0, 1] range, making similarity calculations consistent across practices.

3.3 Collaborative Filtering Algorithm

The collaborative filtering algorithm finds similar teams and uses their improvement patterns:

Step 1: Build Similarity Matrix

- For each team at a target month, calculate practice maturity vector
- Compare target team's vector against all other teams' vectors at all past months
- Use cosine similarity to measure similarity

Step 2: Find K Most Similar Teams

- Select K teams with highest similarity scores; K (5, 10, or 19) and the minimum similarity threshold (0.0, 0.5, or 0.75) are chosen by the global monthly policy (§6.5), not fixed defaults
- Deduplicate to ensure K different teams (not same team at different months)

Step 3: Extract Improvement Patterns

- For each similar team, check which practices showed subsequent observed improvement within a fixed 2-observed-snapshot window after it looked similar to the target team
- Only use improvements that occurred before or at the target month (prevent data leakage)
- Weight improvements by similarity score

Mathematical Formulation:

```
similarity_score(practice) = Σ (similarity_weight × improvement_magnitude)
```

Where:

- Sum is over all similar teams that improved the practice
- similarity_weight is the cosine similarity between teams
- improvement_magnitude is the change in practice score (0-1 range)

3.4 Practice Transition Model Algorithm

The sequence learning algorithm learns transition patterns from historical data:

Step 1: Learn Transition Matrix

- For each team, walk its months chronologically and identify "improvement-bearing" steps — consecutive months where at least one practice improved — skipping over months where nothing improved
- Chain each improvement-bearing step to the *next* one: every practice improved in step *i* gets a transition edge to every practice improved in step *i+1* (a full cross-product between the two sets)

- **Same-month ties**: when multiple practices improve within a single step, no edge is created between them — simultaneous improvements carry no ordering information, so asserting a direction between them would be arbitrary (this was a known limitation of an earlier version, which ordered same-step improvements by their column position in the source spreadsheet)
- Build transition matrix: P(B | A improved) = count(A→B) / Σ_X count(A→X), where count(A→B) is the number of times a practice in some team's improvement-bearing step contained A and the team's *next* improvement-bearing step contained B. The denominator is every observed transition originating from A.

Step 2: Time-Limited Learning

- Only learn from months < current_month (prevent data leakage)
- Learn transition patterns up to the current month
- Cache transition patterns for efficiency

Step 3: Apply Sequence Boost

- Check if target team recently improved any practices, in its own fixed 2 preceding observed snapshots (never tunable)
- For each recently improved practice, boost practices that typically follow it
- Weight boost by transition probability

Mathematical Formulation:

```
sequence_score(practice) = Σ transition_probability(recent_practice → practice)
```

Where:

- Sum is over all recently improved practices
- transition_probability is learned from historical data

3.5 Global Two-Month Adaptive Blend Scoring

The blend combines similarity, sequence, and time-aware popularity signals under one policy selected per prediction month (§6.5), not per team and not fixed in advance:

Step 1: Normalize Each Component

- Similarity and sequence: normalize over all evidence, then restrict to candidate practices (practices not already at maximum maturity for the target team)
- Historical popularity: restrict to candidate practices, then normalize
- Recent popularity (the single most recent organization-wide transition): normalize organization-wide, then restrict to candidate practices

Step 2: Combine with the Month's Selected Weights

```
popularity   = recency_weight × recent_popularity_norm + (1 − recency_weight) × his-
torical_popularity_norm
final_score  = similarity_weight × sim_norm + sequence_weight × seq_norm + popularity_weight × popularity
```

`similarity_weight`, `sequence_weight`, and `popularity_weight` are one of 15 combinations of 0%/25%/50%/75%/100% summing to exactly 100%; `recency_weight` is one of 0%/25%/50%/75%/100%. Both are chosen once per prediction month by the global policy selection described in §6.5 — there is no fixed default and no per-team or per-request override.

Step 3: Filter and Rank

- Filter out practices already at maximum maturity (current_level >= 1.0) — done before scoring, as part of building the candidate set, not as a post-hoc filter on the final scores
- Sort by score (descending), with ties broken deterministically by practice name (rather than by dict/set iteration order, which in Python depends on the process's hash seed and is not reproducible run-to-run) — this ensures the same inputs always produce the same ranked recommendations
- Return the top 2 recommendations for an eligible team-month. A request for any other count is rejected rather than silently honored; when fewer than two candidates remain, return an empty list with an explanatory message

3.6 Validation Methodology (Backtesting)

The validation methodology follows the original proposal's approach:

Rolling Window Backtest:

1. For each month starting from month 4:
    - Train on all months before it (months < test_month)
    - Generate likely next-practice recommendations for that month
    - Validate against actual data for that month

Validation Criteria:

- Compare recommendations against actual improvements in test_month, test_month+1, and test_month+2
- Account for adoption timelines (improvements may occur 1-3 months after recommendation)
- Calculate accuracy: correct_predictions / total_predictions

Random Baseline:

- Calculate probability of getting at least one correct with random selection
- Formula: P(at least one correct) = 1 - C(n-k_avg, top_n) / C(n, top_n)
- Where n = total practices, k_avg = average improvements per case, top_n = recommendations

Time-Aware Popularity Comparison Arm (supplementary):

- A random baseline alone invites the reasonable question of whether the blend beats *any* systematic heuristic, not just chance. As a stronger comparison, the backtest also selects a pure time-aware-popularity policy each month — 0% similarity, 0% sequence, 100% popularity, with only the popularity recency weight chosen — under exactly the same walk-forward rule as the blend itself (§6.5), evaluated on exactly the same evaluable cases
- This is a naive, personalization-free heuristic: it ignores the target team's specific state and recent improvement history entirely, always returning the organization-wide practices that are improving most (subject only to the per-team maxed-out filter)
- Because it is selected under the same monthly rule as the blend (rather than being a single fixed heuristic), it replaces the earlier static popularity baseline used in early research — see §6.5, `src/ml/policy.py`, and `tests/test_blend_reproduction.py`
- Formula: accuracy = correct_predictions_popularity / total_predictions, computed per month (same per-month-averaging convention as the primary Accuracy metric) and compared against the blend's actual accuracy via the same gap/improvement-factor framing used for the random baseline

Improvement Metrics:

- **Accuracy**: Percentage of recommendations that matched actual improvements
- **Improvement Factor**: Accuracy / Random Baseline
- **Improvement Gap**: Accuracy - Random Baseline

Supplementary Rank-Aware Metrics:

The headline Accuracy above (also called Hit Rate@N) is a binary hit/miss per case: it ignores recommendation order and gives full credit even if only 1 of N recommendations was correct. Three stricter, rank-aware metrics are reported alongside it, each with its own matching random baseline:

- **Precision@N**: Correct recommendations ÷ total recommendations made (top_n). Penalizes wrong picks — getting 1 of 2 right scores 0.5 here, vs. 1.0 in Accuracy. Random baseline: k_avg / n.
- **Recall@N**: Correct recommendations ÷ practices actually improved. Measures coverage of a team's real improvement activity; capped at top_n ÷ actual improvements, so a low value can reflect that cap rather than a weaker model. Random baseline: top_n / n.
- **MRR (Mean Reciprocal Rank)**: 1.0 if the first recommendation was correct, 0.5 if the second was the first hit, 0 if none were correct. Rewards ranking the right answer first. Random baseline is computed per case via the exact negative-hypergeometric expectation (not linear in k, so it can't be derived from k_avg like the other two).

These are supplementary diagnostics computed by `BacktestEngine` (backed by `MetricsCalculator.calculate_hit_rate` and `calculate_mrr`) and shown in the Backtest tab, split into primary and sensitivity results (§6.5); they do not change the headline 58.0% primary accuracy / 26.0% random baseline figures reported elsewhere in this document.

Why Hit Rate@N Remains the Headline Metric

It would be reasonable to assume a stricter, rank-aware metric was left out of the headline because it looked worse. The opposite is true. On the same primary backtest scope, Precision@N, Recall@N, and MRR each beat their own random baseline by a factor at least comparable to Hit Rate@N's:

Metric	Improvement Factor vs. Random
Hit Rate@N (headline)	2.23x
Precision@N	2.57x
Recall@N	2.64x
MRR	2.36x

So Hit Rate@N is not the flattering choice among the four — if anything it is the most conservative. It is reported as the headline for a domain-specific reason, not a statistical one: it matches the unit of value that actually matters in agile transformation work.

The constraint on agile adoption is rarely correctness — teams are rarely short on plausible next steps. It is momentum: whether a team acts at all, and keeps acting, cycle over cycle. A recommendation list is not a forced-choice exam that must be graded in full; it is a menu a team uses to pick *one* concrete next step. If a team adopts the one practice on the list that was genuinely a good next move, that hit is enough to validate the recommendation and sustain engagement into the next cycle — regardless of whether the other N-1 items were also correct. Conversely, a list that scores well on Precision@N (say, 2 of 3 items are technically correct) but whose *adopted* item happens to be the wrong one delivers no operational value: nothing changed for the team that cycle. Hit Rate@N is chosen because it operationalizes exactly this — at least one correct, actionable step a team can commit to — which is the mechanism by which the system sustains adoption momentum, not because it is easiest to report favorably.

Precision@N, Recall@N, and MRR remain valuable and are reported alongside it because they answer different questions a reviewer will legitimately ask: how much of the list is wasted effort (Precision@N), how much of a team's real improvement activity the system captures (Recall@N), and whether the system tends to rank the correct answer first (MRR). Those measure list quality. Hit Rate@N measures the operational trigger — whether the recommendation, as consumed by a team picking one thing to try, was worth acting on.

3.7 Worked Examples

This section provides detailed examples showing how the recommendation system works with actual data, demonstrating both similarity-based and sequence-based recommendations.

Note on the weight used below: these worked examples illustrate the score-combination *mechanism* using a fixed illustrative similarity weight of 0.7, matching the shipped system's behavior before the global monthly adaptive blend (§3.5) was introduced. In the current system the similarity/sequence/popularity weights are not fixed at 0.7/0.3 — they are re-selected for every prediction month

from the 675-policy grid described in §6.5. The arithmetic below (weighted sum → normalize → filter maxed-out practices → rank) is otherwise unchanged; only the weight values and the addition of a third (popularity) term differ in production.

Example 1: Similarity-Based Recommendation

Scenario: Team "AADS" at month 200105 (May 2020) needs recommendations for next practices to focus on.

Step 1: Current Team State Team AADS's practice maturity profile at month 200105:

- CI/CD: Level 1 (0.33 normalized)
- Test Automation: Level 0 (0.00 normalized)
- DoD (Definition of Done): Level 3 (1.00 normalized)
- Code Review: Level 2 (0.67 normalized)
- TDD: Level 0 (0.00 normalized)
- ... (other practices)

Step 2: Find Similar Teams The system compares AADS's profile against all teams at all past months (months < 200105). Using cosine similarity, it finds the 19 most similar teams:

Similar Team	Similarity Score	Historical Month	State When Similar
Team B	0.92	200103	CI/CD=1, Test Automation=0, DoD=3, Code Review=2
Team C	0.89	200102	CI/CD=1, Test Automation=0, DoD=3, Code Review=2
Team D	0.87	200104	CI/CD=1, Test Automation=0, DoD=3, Code Review=1
...	...	...	...

Step 3: Extract Improvement Patterns For each similar team, the system checks which practices showed subsequent observed improvement within the next two recorded snapshots (but only using snapshots ≤ 200105 to prevent data leakage):

Team B (similarity: 0.92, at month 200103):

- Improved "Test Automation" from 0 to 1 in month 200104 (improvement magnitude: 0.33)
- Improved "CI/CD" from 1 to 2 in month 200105 (improvement magnitude: 0.33)

Team C (similarity: 0.89, at month 200102):

- Improved "Test Automation" from 0 to 1 in month 200103 (improvement magnitude: 0.33)
- Improved "CI/CD" from 1 to 2 in month 200104 (improvement magnitude: 0.33)

Team D (similarity: 0.87, at month 200104):

- Improved "Test Automation" from 0 to 1 in month 200105 (improvement magnitude: 0.33)

Step 4: Calculate Similarity Scores For each practice, sum weighted improvements from similar teams. Each improvement is weighted by the cosine similarity score between the target team and the similar team:

Test Automation:

- Team B (similarity: 0.92): 0.92 × 0.33 = 0.304
- Team C (similarity: 0.89): 0.89 × 0.33 = 0.294
- Team D (similarity: 0.87): 0.87 × 0.33 = 0.287
- **Total similarity score: 0.885**

CI/CD:

- Team B (similarity: 0.92): 0.92 × 0.33 = 0.304
- Team C (similarity: 0.89): 0.89 × 0.33 = 0.294
- **Total similarity score: 0.598**

Note: The similarity scores (0.92, 0.89, 0.87) are cosine similarity values between teams, not to be confused with the similarity_weight parameter (0.7) used later for combining similarity and sequence scores.

Step 5: Sequence Scores Team AADS did not recently improve any practices (no sequence boost applies in this example).

Step 6: Normalize and Combine

- Normalize similarity scores:
  - Test Automation: 0.885 / 0.885 = 1.000
  - CI/CD: 0.598 / 0.885 = 0.676
- Sequence scores: 0.000 (no recent improvements)
- Combined scores (similarity_weight = 0.7):
  - Test Automation: (1.000 × 0.7) + (0.000 × 0.3) = 0.700
  - CI/CD: (0.676 × 0.7) + (0.000 × 0.3) = 0.473
- Final normalization (normalize combined scores):
  - Test Automation: 0.700 / 0.700 = 1.000
  - CI/CD: 0.473 / 0.700 = 0.676

Step 7: Filter and Rank

- Filter out practices at max level (DoD is already at Level 3, excluded)
- Rank by final normalized score:
  1. **Test Automation**: 1.000
  2. **CI/CD**: 0.676

Recommendation:

- **Top Recommendation**: Test Automation (score: 1.000)
  - Why: "3 similar team(s) improved this practice"
  - 3 teams (B, C, D) with 87-92% similarity improved Test Automation

Validation Result: Team AADS actually improved Test Automation from Level 0 to Level 1 in month 200106, confirming the recommendation was successful.

Example 2: Sequence-Based Recommendation

The Practice Transition Model does not encode a preselected agile-practice pathway. For a team with recent observed improvements, it retrieves the practices that most often improved at the next improvement-bearing step in the organization's history, then combines those conditional frequencies with similarity-based scores. The empirical transition table in §6.8 shows the current full-data evidence; during backtesting, the same calculation is learned only from data before the evaluation month. A transition frequency is organizational evidence, not a causal rule or a guarantee for an individual team.

Key Insights from Examples:

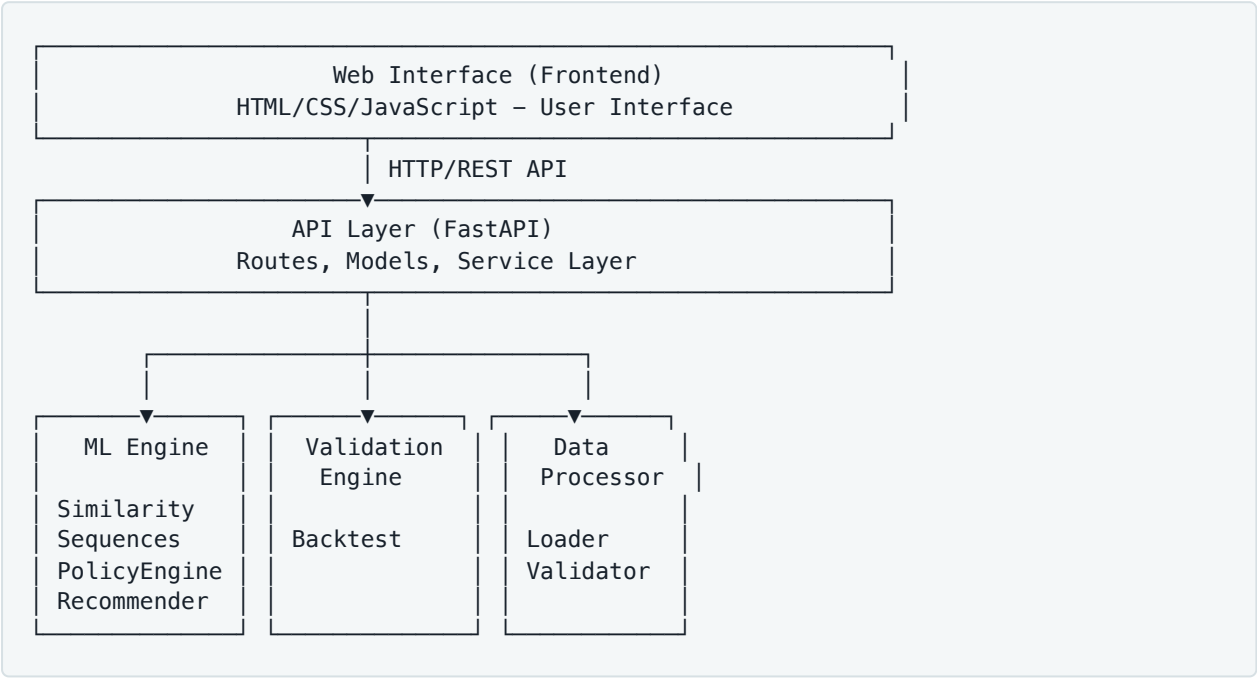
1. **Similarity-Based Recommendations** work best when:
   - Many similar teams have improvement history
   - Similar teams show clear improvement patterns
   - Target team's profile matches historical patterns
2. **Sequence-Based Recommendations** work best when:
   - Team recently improved practices
   - Strong sequence patterns exist in organizational data
   - Transition evidence is considered alongside the team's current maturity profile
3. **Hybrid Approach** combines both signals:
   - When both similarity and sequence agree, confidence is high
   - When they differ, the weighted combination provides balanced recommendations
   - Normalization ensures both signals contribute proportionally

---

4. System Design and Architecture

4.1 High-Level Architecture

The system is built using a modular architecture with clear separation of concerns:



4.2 Component Descriptions

Data Module (`src/data/`):

- **DataLoader**: Loads Excel files and extracts team/practice/month data
- **DataProcessor**: Normalizes data, builds team histories indexed by month
- **DataValidator**: Validates data quality, checks for missing values
- **PracticeDefinitionsLoader**: Loads practice level definitions from Excel

ML Module (`src/ml/`):

- **SimilarityEngine**: Calculates cosine similarity between teams, finds K most similar teams
- **SequenceMapper**: Learns the practice transition matrix and organization-wide popularity counts from historical data
- **PolicyEngine**: Owns the global monthly policy grid, cohort building, and blend scoring
- **RecommendationEngine**: Thin wrapper delegating to `PolicyEngine`, kept for a stable constructor shape

Validation Module (`src/validation/`):

- **BacktestEngine**: Runs the rolling window backtest of the blend, split into primary and sensitivity aggregates

- **Metrics**: Calculates accuracy, improvement factors, random baselines

API Module (`src/api/`):

- **Routes**: FastAPI route definitions for REST endpoints
- **Service**: Service layer wrapping ML components for API
- **Models**: Pydantic models for request/response validation

Interface Module (`src/interface/`):

- **CLI**: Command-line interface for interactive use
- **Formatter**: Formats output for display

4.3 Data Flow

1. **Data Loading**:
    - Excel file → DataLoader → DataFrame
    - DataFrame → DataValidator → Validated DataFrame
    - Validated DataFrame → DataProcessor → Team Histories
2. **Recommendation Generation**:
    - Team Name + Month → SimilarityEngine → Similar Teams
    - Similar Teams → RecommendationEngine → Similarity Scores
    - Historical Data → SequenceMapper → Transition Matrix
    - Recent Improvements → SequenceMapper → Sequence Scores
    - Similarity Scores + Sequence Scores → RecommendationEngine → Final Recommendations
3. **Validation**:
    - Historical Data → BacktestEngine → Per-Month Recommendations
    - Recommendations + Actual Data → BacktestEngine → Accuracy Metrics

4.4 API Design (REST API with FastAPI)

The system exposes a REST API using FastAPI:

Endpoints:

- `GET /api/teams` - Get all teams with metadata
- `GET /api/teams/with-improvements` - Get teams/months where improvements occurred
- `GET /api/teams/{team_name}/months` - Get available months for a team
- `POST /api/recommendations` - Get recommendations for a team (`top_n` pinned to 2)
- `POST /api/backtest` - Run the backtest of the global monthly adaptive blend (no parameters)
- `GET /api/stats` - Get system statistics

- `GET /api/sequences` - Get learned improvement sequences
- `GET /api/example-data` - Serve the raw Excel dataset file for in-browser preview
- `GET /api/docs` - Serve project documentation content

There is no static all-history parameter optimizer or `/api/optimize*` family of endpoints - the global monthly policy (§6.5) is the sole configuration authority for the primary flow and its backtest.

Request/Response Format:

- JSON format for all requests and responses
- Pydantic models ensure type safety and validation
- Error responses include detailed error messages

4.5 Frontend Architecture

The web interface is built with vanilla HTML/CSS/JavaScript using a Dark Academic Research Lab visual theme:

Structure:

- Single-page application with tabbed interface
- Four main tabs (in order): Statistics, Backtest Validation, Sequences, Recommendations
- Statistics is the default active tab on load
- Dynamic content loading via JavaScript fetch API
- Real-time updates without page refresh

Key Features:

- Team selection and a `Current Month → Prediction Month` dropdown; each option makes the baseline snapshot and target month explicit
- Interactive result displays with inline tooltip explanations on each tab
- A policy audit box showing the selected policy's weights, peer pool, and popularity recency for the current prediction month - no configuration form, since there are no user-adjustable model parameters
- Primary and sensitivity result sections in the Backtest tab, with estimated progress while the run is in progress
- "About" modal that renders project documentation in-browser

---

5. Implementation

5.1 Technology Stack and Rationale

Python 3.10+ (vs. Java/C++/C# from proposal):

- **Rationale**: Python was chosen over the languages mentioned in the proposal due to:
  - Rich ecosystem for data science and machine learning (pandas, numpy, scikit-learn)
  - Rapid prototyping capabilities
  - Excellent Excel file handling (openpyxl)
  - Modern web framework (FastAPI) with async support
  - Strong community support and documentation
  - Easier integration with data analysis tools

Key Libraries:

- **pandas**: Data manipulation and Excel file reading
- **numpy**: Numerical operations and array handling
- **scikit-learn**: Cosine similarity calculations
- **scipy**: Statistical functions (combinations for random baseline)
- **openpyxl**: Excel file format support
- **fastapi**: Modern, fast web framework for building APIs
- **uvicorn**: ASGI server for FastAPI
- **pydantic**: Data validation using Python type annotations

5.2 Key Implementation Decisions

1. Cross-Temporal Similarity Matching:

- Compares target team at current month against all teams at all past months
- Leverages all available historical data for better recommendations
- Deduplicates to ensure K different teams (not same team at different months)

2. Time-Limited Sequence Learning:

- Sequences learned only from months < current_month
- Prevents data leakage in backtesting
- Uses caching to avoid recomputation

3. Sliding Window Validation:

- Rolling window approach: train on past, test on future

- Uses global prediction months starting at the fourth recorded month; each team uses its latest available baseline strictly before that month
- Validates against the baseline's next three observed snapshots

4. Data Leakage Prevention:

- Recommendation evidence is bounded at the team's baseline: comparable snapshots and sequence history are strictly earlier, and peer look-ahead cannot pass that baseline
- Future months used only for validation, never to generate recommendations
- Explicit checks prevent using future data in recommendations

5. Normalization Strategy:

- Normalize similarity and sequence evidence before masking to candidates; historical popularity is masked then normalized, while recent popularity is normalized organization-wide then masked
- Combine the three components with the selected policy weights; final blended scores are ranking values and are not re-normalized

5.3 Code Organization

The source code is organized across 23 Python modules:

Module Structure:

```
src/
├── data/           # Data loading, processing, validation (~600 lines)
├── ml/             # Machine learning algorithms (~1,200 lines)
├── validation/     # Backtest validation of the blend (~700 lines)
├── api/            # Web API layer (~600 lines)
├── interface/      # CLI interface (~400 lines)
└── web_main.py     # Web server entry point (~200 lines)
```

Design Patterns:

- **Separation of Concerns**: Each module has a single responsibility
- **Dependency Injection**: Components receive dependencies through constructors
- **Service Layer**: API service wraps ML components for clean API interface
- **Factory Pattern**: Route creation functions for API setup

5.4 Performance Considerations

Performance and Scalability:

- Efficient data structures: Team histories stored as dictionaries indexed by month
- Caching: Sequence learning results cached to avoid recomputation
- Vectorized operations: NumPy arrays for efficient similarity calculations
- Lazy evaluation: Similarity matrices built on-demand, not pre-computed

Optimization Strategies:

- Estimated progress: the interface reports progress while a backtest is running
- Cached policy scoring: `PolicyEngine` caches case components, evaluable cohorts, and per-month hit-rate sweeps so repeated backtests and recommendation calls for the same month are near-instant after the first pass
- Async operations: FastAPI async endpoints for concurrent request handling
- Memory efficiency: Process data in chunks where possible

5.5 Real-World Data Integration

Excel Format Support:

- Reads standard Excel files (.xlsx) as specified in proposal
- Flexible column detection: Automatically identifies practice columns
- Error handling: Graceful handling of missing data, invalid formats
- Practice definitions: Optional Excel file for practice level definitions

Data Validation:

- Checks for required columns (Team Name, Month)
- Validates data types and ranges
- Handles missing values (fills with 0, normalized to 0.0)
- Reports data quality issues

---

6. Evaluation and Results

6.1 Dataset Description

The system was evaluated on real organizational data:

- **Teams**: 87 teams participating in agile adoption
- **Practices**: 35 different agile practices tracked
- **Time Period**: 10 months of historical data
- **Observations**: 655 total observations (team-month combinations)
- **Data Format**: Excel matrices with teams × practices × maturity levels (0-3)

The 655 observations do not represent uniform coverage for every team: 48 teams have all 10 recorded months, while 39 teams have partial coverage ranging from 1 to 9 months. This variation occurs because teams joined or left the recorded population at different points during the ten-month period. Analyses use each team's available chronological history and do not assume that every team is observed in every month.

Recorded months per team	Teams
1	2
2	7
3	8
4	5
5	3
6	5
7	5
8	1
9	3
10	48

This dataset aligns with the proposal's scale (70+ teams, 30+ practices) and represents a realistic large-scale agile transformation scenario.

6.2 Evaluation Metrics

Primary Metrics:

- **Accuracy**: Percentage of recommendations that matched actual improvements
- **Random Baseline**: Expected accuracy with random practice selection

- **Improvement Factor**: Accuracy / Random Baseline (how many times better than random)
- **Improvement Gap**: Accuracy - Random Baseline (absolute improvement)

Secondary Metrics:

- **Per-Month Accuracy**: Accuracy broken down by validation month
- **Teams Tested**: Number of teams included in validation
- **Total Recommendations Evaluated**: Number of recommendation cases evaluated
- **Average Improvements per Case**: Average number of practices improved per team-month

Supplementary Rank-Aware Metrics:

- **Precision@N**: Correct recommendations ÷ recommendations made — penalizes wrong picks
- **Recall@N**: Correct recommendations ÷ practices actually improved — measures coverage
- **MRR**: Mean Reciprocal Rank — rewards ranking the correct recommendation first

Each has its own matching random baseline, gap, and improvement factor, reported alongside the primary Accuracy metrics (see §3.6 for definitions and baseline formulas).

6.3 Backtest Results

The backtest reports two scopes, never mixed together: **primary** covers the five prediction months whose 3-snapshot outcome window has fully closed against the dataset's end; **sensitivity** covers all seven prediction months, including the two with a truncated outcome window.

Primary Performance (5 months, 121 evaluable cases):

- **Accuracy (Hit Rate@N)**: 58.0%
- **Random Baseline**: 26.0%
- **Improvement Factor**: 2.2x better than random
- **Time-Aware Popularity Comparison**: 55.7% (blend +2.3 percentage points)

Sensitivity Performance (all 7 months, 151 evaluable cases):

- **Accuracy (Hit Rate@N)**: 50.9%
- **Random Baseline**: 23.5%
- **Improvement Factor**: 2.2x better than random
- **Time-Aware Popularity Comparison**: 47.5% (blend +3.4 percentage points)

Per-Month Results (primary and sensitivity):

Month	Evaluable Cases	Blend Accuracy	Time-Aware Popularity	Scope
2020-05-03	21	28.6%	28.6%	Primary (bootstrap policy)
2020-06-08	22	63.6%	63.6%	Primary (bootstrap policy)
2020-07-05	27	66.7%	66.7%	Primary (bootstrap policy)

Month	Evaluable Cases	Blend Accuracy	Time-Aware Popularity	Scope
2020-08-03	24	79.2%	75.0%	Primary
2020-09-06	27	51.9%	44.4%	Primary
2020-10-05	24	33.3%	20.8%	Sensitivity only (truncated window)
2020-11-04	6	33.3%	33.3%	Sensitivity only (truncated window)

Three of the five primary months use the **bootstrap policy** (100% popularity, 50/50 recency) because no earlier prediction month yet had a completed 3-snapshot outcome window - in those months the blend and the time-aware-popularity comparison arm are identical by construction. Only 2020-08-03 and 2020-09-06 exercise a genuinely mixed policy, and both show the blend ahead.

Supplementary Rank-Aware Metrics (primary scope):

Metric	Value	Random Baseline	Improvement Factor
Precision@N	35.6%	13.9%	2.57x
Recall@N	17.6%	6.7%	2.64x
MRR	0.46	0.19	2.36x

Each stricter, rank-aware metric shows an improvement factor over its own random baseline that meets or exceeds Hit Rate@N's 2.23x — see §3.6 for why Hit Rate@N is still reported as the headline metric despite this.

How to read the time-aware-popularity comparison:

This is an exploratory result, not a claim of proven superiority over popularity alone — the comparison arm is itself selected under the same monthly walk-forward rule as the blend (restricted to 0% similarity / 0% sequence), not a single fixed heuristic, and three of five primary months tie exactly because both arms fall back to the same bootstrap policy. The remaining +2.3 percentage-point primary margin is an aggregate organizational backtest result (a macro-average across the tested months), not a guarantee of improvement for every individual team or month.

This is an important, honest framing: most of the blend's advantage over pure random selection is attributable to organization-wide improvement trends that even a naive, non-personalized popularity heuristic captures. The blend's specific value-add — from personalizing to each team's own state via collaborative filtering and sequence evidence, on top of time-aware popularity — is the smaller remaining margin shown above, not the larger margin over random chance. Both comparisons are reported because they answer different questions: random baseline establishes the problem isn't trivial to solve by chance; the time-aware-popularity arm establishes how much value personalization specifically adds on top of a properly time-aware "know the organization's current trends" heuristic. The executable policy and its reproducible reference assertions are in `src/ml/policy.py` and `tests/test_blend_reproduction.py`.

6.4 Validation Methodology Results

The validation methodology follows the original proposal:

- **Training**: Uses data from months before the test month
- **Recommendation generation**: Generates likely next practices for the test month
- **Validation**: Compares recommendations against actual improvements in test month, test_-month+1, and test_month+2
- **Success Criteria**: At least one recommended practice actually improved in the validation window

Results demonstrate that the system identifies likely next practices with meaningful accuracy, validating the approach proposed in the original project proposal.

6.5 Global Monthly Policy Selection

There is no static, all-history parameter optimizer. An earlier version of the system had one (`OptimizationEngine`, a grid search over fixed default parameters selected once from *all* historical months), but an early popularity-baseline investigation found that its headline figures were selected on the same data they were measured on: a legitimate walk-forward search over the same parameter space actually landed at 40.8-42.6% accuracy, below a plain organization- wide popularity baseline (44.5%). That optimizer, its three `/api/optimize*` endpoints, its web controls, and its CLI menu options were removed entirely, not hidden.

In its place, one **global policy** is selected automatically for each prediction month:

The 675-Combination Grid:

- Peer count: 5, 10, or 19 similar teams
- Minimum similarity threshold: 0.0, 0.5, or 0.75
- Similarity / sequence / popularity weight triple: 15 combinations of 0%/25%/50%/75%/100% summing to exactly 100%
- Popularity recency weight: 0%, 25%, 50%, 75%, or 100%
- 3 × 3 × 15 × 5 = 675 candidate policies

Selection Rule:

- For a target prediction month, only earlier prediction months whose full 3-snapshot outcome window has already closed are used as evidence - never the target month's own outcome, and never any later month's
- The policy maximizing mean Hit Rate@N across those completed months is selected; ties are broken deterministically (prefer more popularity-heavy, then lower recency, then lower similarity weight, then lower sequence weight, then lower peer count, then lower similarity threshold)
- When no prior prediction month yet has a completed outcome window, the **bootstrap policy** applies: 100% popularity, 50% recent / 50% historical recency weighting
- The same selected policy is replayed identically by the web interface, the CLI, and the backtest for a given prediction month - there is no per-team or per-request override

Component windows are fixed, never part of the grid: both the similarity look-ahead (how far past a peer's similar-looking snapshot to check for improvements) and the sequence recency window (how far back to check the target team's own recent improvements) are fixed at exactly 2 observed snapshots. See §6.3 for the resulting per-month policy and accuracy figures.

6.6 Performance Analysis

Computational Performance:

- **Recommendation Generation**: well under 1 second per team once a month's policy and cached case components are warm
- **Backtest Validation**: on the order of tens of seconds for the full dataset (dominated by the first-time sweep of all 675 policies per prediction month; subsequent requests reuse `PolicyEngine`'s caches)

Scalability:

- Handles 87 teams × 35 practices × 10 months efficiently
- Memory usage: ~100-200 MB for full dataset
- Can scale to larger datasets with same architecture

Accuracy vs. Speed Trade-offs:

- A larger peer count in the grid improves candidate coverage but increases per-policy computation
- Sequence and case-component caching reduces repeated computation across prediction months
- The interface reports estimated progress while a backtest is running

6.7 Practical Validation Readiness

The system is ready for real-world testing as proposed in the original project timeline (May-July):

Deployment Readiness:

- **Web Interface**: Fully functional, user-friendly interface
- **API Endpoints**: Complete REST API for integration
- **Data Integration**: Excel file format as specified in proposal
- **Error Handling**: Robust error handling and validation
- **Documentation**: Complete user and technical documentation

Testing Capabilities:

- Can be deployed with selected teams for pilot testing
- Supports real-time recommendations based on current data
- Validation framework can evaluate real-world implementation results
- Results can be compared against historical recommendation outcomes

6.8 Learned Improvement Sequences

This section presents the observed improvement transitions learned from all available months in the dataset. They are descriptive organizational patterns, not prescribed adoption pathways.

Analysis Methodology:

- Sequences learned from all teams' improvement history across all months
- The model retains the 30 practices that pass the project's missing-data filter
- Transition matrix built from consecutive improvement-bearing steps; same-step improvements are not assigned an order
- Probabilities calculated as: P(B | A improved) = count(A→B) / Σ_X count(A→X)
- Rows sorted by observed transition count

Top Observed Transitions:

The following table was recomputed from `combined_dataset.xlsx` with the production loader, missing-data filter, processor, and Practice Transition Model. It lists the ten highest-count transitions among 471 observed transitions and 310 unique practice pairs.

From Practice	To Practice	Observed Count	All Transitions from Source	Conditional Frequency
Unified backlog	Product Owner	7	24	29.2%
Scrum Master	Tech debt strategy	7	28	25.0%
Tech debt strategy	Product Owner	6	47	12.8%
Reducing WIP	Tech debt strategy	5	15	33.3%
Scrum Master	Unified backlog	5	28	17.9%
DoR	Reducing WIP	5	30	16.7%
Retro	Scrum Master	5	38	13.2%
Tech debt strategy	Tech debt strategy	5	47	10.6%
DoR	Tech debt strategy	4	30	13.3%
DoD	Tech debt strategy	4	32	12.5%

The denominator is shown because a high percentage based on a small number of outgoing transitions is not necessarily stronger evidence than a lower percentage with more observations. For example, the table is ranked by count rather than conditional frequency to make that evidence visible.

Sequence Statistics:

- **Practices with observed outgoing transitions**: 30
- **Unique practice-to-practice pairs**: 310
- **Total transition counts**: 471

- **Most frequently improved practice**: Tech debt strategy (34 improvement-bearing steps)
- **Average unique follow-on practices per source practice**: 10.3

Key Insights:

1. **Organization-specific evidence**: The table reports what was observed in this organization's recorded history; it does not assert that a named practice inherently enables another practice.
2. **Multiple pathways**: Each source practice can have several observed follow-on practices, which reflects differing team contexts and priorities.
3. **Descriptive, not causal**: Transitions guide ranking as empirical evidence only. They do not prove causation or prescribe an order for every team.

Practical Implications:

- **Guided Progression**: Teams can use these sequences as evidence-informed improvement pathways
- **Readiness Indicators**: If a team improved Practice A, practices that typically follow A may be useful candidates to consider
- **Risk Reduction**: Following organizational patterns may reduce risk compared to random practice selection
- **Customization**: While patterns exist, teams can still choose alternative paths based on their specific needs

Limitations:

- Sequences are probabilistic, not deterministic - not all teams follow the same path
- Patterns reflect organizational context - may differ for other organizations
- Sequences learned from historical data may not account for future changes
- Some sequences may be correlation rather than causation

6.9 Maximum-Maturity Analysis

The maturity scale has a fixed upper bound: level 3 (normalized value 1.0). Once a team has reached level 3 for a practice, no further maturity improvement can be observed for that team–practice pair within this scale. This matters for both recommendations and evaluation: a level-3 practice is excluded from the candidate recommendation set, rather than being treated as an opportunity for further improvement (see §3.5, Step 4).

Analysis basis: This analysis uses the latest available profile for each of the 87 teams in `combined_dataset.xlsx`, across all 35 practices. Seventy-five teams have a latest profile dated 2020-11-04; the remaining 12 teams have earlier latest records. The analysis therefore provides the most complete per-team maturity snapshot available in the supplied dataset, rather than a single common-month snapshot.

Team-level distribution of practices at maximum maturity:

Share of practices at level 3	Teams	Share of teams
0–20%	74	85.1%
>20–50%	11	12.6%
>50–80%	2	2.3%
>80%	0	0.0%

Overall, 78 of 87 teams (89.7%) reached level 3 in at least one practice. Teams reached level 3 in an average of 3.84 of the 35 practices; the observed range was 0 to 22 practices. These figures show that maximum maturity is common in selected practices, but broad saturation across a team's full practice set is uncommon.

Maximum-maturity prevalence by practice:

Practice	Teams at level 3	Share of teams
AIM JIRA structure	66	75.9%
Demo	29	33.3%
Unified backlog	27	31.0%
Release tracker	25	28.7%
Scrum area	22	25.3%
Retro	18	20.7%
Defect management strategy	16	18.4%
DoD	14	16.1%
Single branch strategy	14	16.1%
Tasking	13	14.9%
DoR	11	12.6%
Scrum Master	10	11.5%
Sprint burndown	8	9.2%
Tech debt strategy	8	9.2%
Backlog grooming (sprint)	7	8.0%
Engineering 360	7	8.0%
Product Owner	7	8.0%
Story Points	6	6.9%
Test automation	5	5.7%

Practice	Teams at level 3	Share of teams
AIM ceremonies	4	4.6%
Story mapping	4	4.6%
Customer engagement	3	3.4%
Personas	3	3.4%
Shift-left adoption	3	3.4%
Reducing WIP	2	2.3%
Backlog grooming (release)	1	1.1%
Time to Value Delivery	1	1.1%
BDD	0	0.0%
CI/CD	0	0.0%
Multi component team	0	0.0%
Multi function team	0	0.0%
Spikes template	0	0.0%
TDD	0	0.0%
Tech story template	0	0.0%
User story template	0	0.0%

The distribution is concentrated in a small number of practices, particularly AIM JIRA structure. Practices with no teams at level 3 are not evidence that they are unimportant; they indicate that the supplied dataset contains remaining maturity headroom for those practices. Conversely, a high maximum-maturity count does not imply that every team has completed that practice, only that the model must not recommend it again to teams that already reached level 3.

---

7. Discussion

7.1 How Implementation Addresses Proposal Objectives

The implemented system successfully addresses all objectives stated in the original proposal:

1. Automated Recommendations:

- Generates exactly two ranked practices for each eligible team
- Based on organizational history up to current point in time
- The recommendation count is fixed; the monthly policy chooses model settings, not callers

2. Machine Learning Application:

- Applies collaborative filtering to find similar teams
- Uses the Practice Transition Model to identify improvement patterns
- Processes the project dataset efficiently

3. Validation:

- Uses historical backtesting methodology as proposed
- Compares recommendations against actual improvements
- Demonstrates 58.0% primary aggregate backtest accuracy with a 2.2x improvement over the random baseline (26.0%), and a 2.3 percentage-point edge over an independently walk-forward-selected time-aware-popularity comparison arm (55.7%); this is exploratory and not a per-team guarantee (§6.3)

4. Practical Deployment:

- System is a functional prototype, ready for pilot testing with selected teams (see §7.3 for the gap to a hardened production deployment)
- Web interface enables easy use by non-technical users
- Ready for real-world testing with selected teams

7.2 Strengths

Technical Strengths:

- **Hybrid Approach**: Combines collaborative filtering and sequence learning for robust recommendations
- **Data Leakage Prevention**: Careful implementation ensures no future data leakage
- **Scalability**: Efficient algorithms support growth beyond the current project dataset
- **Modular Architecture**: Clean separation enables maintenance and extension

Practical Strengths:

- **User-Friendly Interface**: Web interface makes system accessible to non-technical users
- **Auditable monthly selection**: The response exposes the automatically selected policy and the completed months that informed it
- **Real-World Ready**: Excel format matches organizational data collection methods
- **Validation Framework**: Comprehensive backtesting validates approach

7.3 Limitations

Data Limitations:

- A public web/API request needs a team snapshot in a valid global prediction month, a usable baseline before it, and at least two non-maxed candidate practices
- Accuracy depends on data quality and completeness
- May not account for external factors (organizational changes, market conditions)

Algorithm Limitations:

- Recommendations are probabilistic, not deterministic guarantees
- Assumes historical patterns will continue (may not account for paradigm shifts)
- Similarity matching may not capture all relevant team characteristics

Practical Limitations:

- Requires regular data updates (monthly) to maintain accuracy
- Initial setup requires data collection and validation
- The current policy grid and tie-break rule are fixed in code; adapting them to another organization requires an evaluated implementation change

Production-Readiness Limitations:

A working web interface and REST API demonstrate the system end-to-end, but do not by themselves make a system production-ready. The following are not yet implemented and would be required before deployment beyond a pilot with a small number of trusted teams:

- **No authentication/authorization**: the API and UI have no login, access control, or per-team data isolation — anyone with network access can query any team's data
- **No monitoring or alerting**: no logging/metrics infrastructure to detect failures, performance regressions, or data quality issues in production
- **No automated data-refresh pipeline**: the monthly Excel update is a manual step; there is no ingestion pipeline, schema validation on upload, or scheduled retraining
- **Single-process, single-user assumption**: no concurrency handling, rate limiting, or multi-tenant isolation for simultaneous use by multiple analysts
- **No deployment infrastructure**: no containerization, CI/CD pipeline, backup/recovery strategy, or horizontal scaling story beyond a single local/server process

This system is best characterized as a **functional prototype validated on real data and ready for pilot testing with a small number of teams under supervision**, not a production system in the infrastructure-hardening sense of the term.

7.4 Practical Implications for Large Organizations

Business Value:

- **Decision Support**: Provides data-driven recommendations instead of intuition
- **Scalability**: Can serve 70+ teams simultaneously (vs. 1-2 teams manually)
- **Consistency**: Standardized approach across all teams
- **Efficiency**: Estimated to eliminate 4-8 hours/month of manual analysis per team (see basis below)

Basis for the 4-8 hours/month estimate: This figure is a practitioner estimate, not a measurement from a controlled evaluation of this system. It is based on the author's 6 years of experience as an agile coach, corroborated by estimates from 4 fellow agile coaches, reflecting the typical time spent per team per month (a) selecting which agile practice to focus on next and (b) coaching on practices that ultimately turned out to be the wrong choice — i.e., adopted but with zero measurable adoption impact ("zero hit"). No time-motion study or before/after measurement with the actual system deployed has been conducted; this should be read as an informed estimate of the problem's scale from direct practitioner experience, not as an empirically validated result of this specific tool.

Organizational Impact:

- **Faster Transformation**: Can help teams focus on practices with higher estimated success probability
- **Reduced Waste**: Can reduce recommendations of practices teams may not be ready for
- **Learning**: System learns from all teams' experiences, not just individual team history
- **Continuous Improvement**: Gets smarter each month as more data accumulates

7.5 Comparison with Baseline

Random Baseline:

- Random practice selection achieves ~26.0% primary accuracy (per-month average improvements per case and number of recommendations, macro-averaged across months to match accuracy's own aggregation)
- System achieves 58.0% primary accuracy, representing a 2.2x improvement over random, and a 2.3 percentage-point edge over an independently selected time-aware-popularity comparison arm (55.7%)

These are aggregate organizational backtest results, not guaranteed outcomes for each team or month, and the comparison against time-aware popularity remains exploratory (§6.3). Individual results can differ based on a team's history, maturity profile, and subsequent improvements.

Manual Analysis Baseline:

- Manual analysis can serve 1-2 teams per coach per month
- System can serve all 70+ teams simultaneously
- Manual analysis is subjective and inconsistent
- System provides standardized, evidence-based recommendations

7.6 Dataset Scale and Efficiency

The project uses a moderate-sized organizational dataset: 655 recorded team-month rows across 87 teams and 10 global months, with 35 tracked practices. This yields 22,925 team-practice cells before missing-data filtering. A fully populated 87 × 35 × 10 rectangular grid would contain 30,450 cells, but the supplied data is not rectangular because 39 teams have partial month coverage (§6.1). The system processes the recorded dataset efficiently and can support future growth.

Efficiency:

- Processes 655 recorded team-month rows × 35 raw practices in seconds
- Memory-efficient data structures
- Caching reduces redundant computations

Scalability:

- Architecture supports larger datasets (more teams, practices, months)
- Algorithms scale linearly with data size
- Can handle real-time updates as new data arrives

Practical Application:

- Handles data volumes that are impractical for manual analysis
- Processes monthly updates efficiently
- Supports continuous learning as data accumulates

---

8. Conclusions and Future Work

8.1 Summary of Achievements

This project successfully implements an empirical organizational-learning approach for identifying likely large-scale agile implementation pathways, achieving the following:

Its core contribution is deriving team-specific recommendations from observed organizational behavior: peer-team maturity histories, practice-transition patterns, and organization-wide popularity trends, blended under a policy selected automatically per prediction month. The collaborative filtering, Practice Transition Model, time-aware popularity, and global monthly blend mechanism are established implementation techniques used to operationalize that contribution, rather than the novelty claim itself.

Technical Achievements:

- Implemented collaborative filtering algorithm for finding similar teams
- Implemented the Practice Transition Model for identifying improvement patterns
- Implemented time-aware popularity and a global monthly policy selection mechanism, replacing a static all-history parameter optimizer that walk-forward analysis showed to be unreliable (§6.5)
- Achieved 58.0% primary aggregate recommendation alignment, 2.2x better than the random baseline (26.0%), and 2.3 percentage points ahead of an independently selected time-aware-popularity comparison arm (55.7%); individual team outcomes may differ, and this remains an exploratory result (§6.3)

Practical Achievements:

- Built a functional web interface, ready for pilot use (see §7.3 for the gap to production hardening)
- Validated approach using historical backtesting methodology
- Demonstrated scalability for large organizations (87 teams, 35 practices, 10 months)
- Created comprehensive documentation for deployment and maintenance

Alignment with Proposal:

- All original proposal objectives have been met
- System ready for real-world testing as planned
- Excel data format matches organizational requirements
- Validation methodology follows proposed approach

8.2 Real-World Deployment Readiness

The system is ready for deployment and real-world testing:

Deployment Requirements Met:

- Web interface functional and user-friendly

- API endpoints available for integration
- Data format matches organizational Excel files
- Error handling and validation robust
- Documentation complete

Testing Readiness:

- System can be deployed with selected teams for pilot testing
- Real-time recommendations based on current data
- Validation framework can evaluate implementation results
- Results can be compared against recommendation outcomes

Next Steps for Deployment:

1. Select pilot teams for initial testing
2. Deploy system with current organizational data
3. Monitor recommendations and actual improvements
4. Evaluate success/failure alignment
5. Iterate based on feedback and results

8.3 Future Improvements

Algorithm Enhancements:

- **Deep Learning**: Explore neural networks for more complex pattern recognition
- **Contextual Factors**: Incorporate external factors (team size, product type, technology stack)
- **Temporal Patterns**: Better handling of seasonal or cyclical patterns
- **Multi-Objective Optimization**: Consider multiple objectives (speed, quality, cost) simultaneously

Feature Additions:

- **Confidence Intervals**: Provide confidence levels for recommendations
- **What-If Analysis**: Allow users to simulate different scenarios
- **Team Clustering**: Identify teams with similar characteristics automatically
- **Practice Dependencies**: Explicitly model dependencies between practices

System Enhancements:

- **Real-Time Updates**: Stream processing for continuous data updates
- **Distributed Processing**: Scale to very large organizations (1000+ teams)
- **Mobile Interface**: Mobile app for on-the-go access
- **Integration**: Connect with project management tools (Jira, Azure DevOps)

8.4 Potential Extensions

Research Directions:

- **Causal Inference**: Determine causal relationships, not just correlations
- **Transfer Learning**: Apply patterns learned from one organization to another
- **Explainable AI**: Better explanations for why practices are recommended
- **Multi-Organization Learning**: Learn from multiple organizations simultaneously

Practical Extensions:

- **Practice Library**: Expand to include more agile practices and frameworks
- **Custom Metrics**: Allow organizations to define custom practices and metrics
- **Reporting**: Advanced reporting and analytics dashboards
- **Notifications**: Automated alerts when teams should focus on specific practices

Academic Extensions:

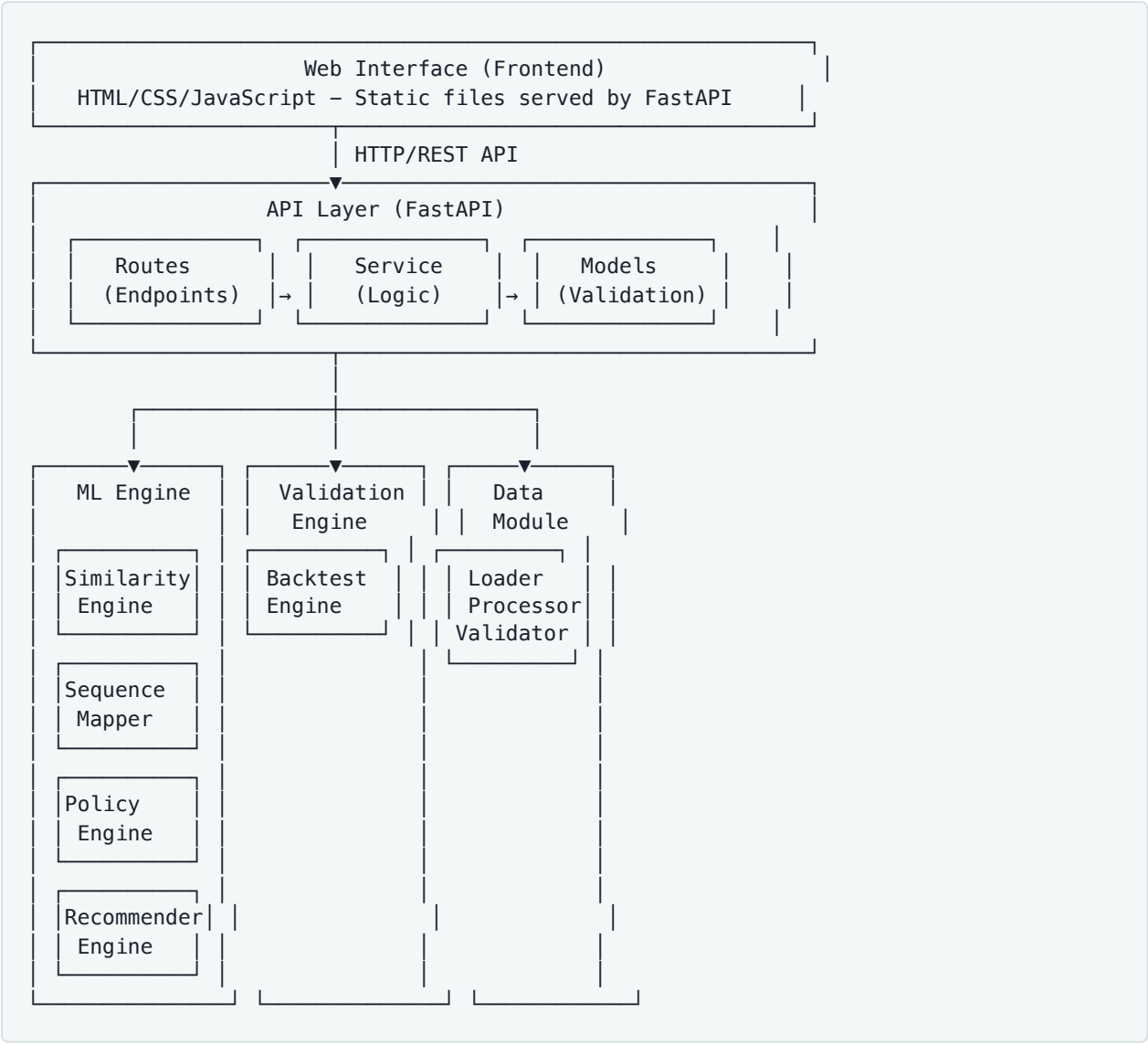
- **Publication**: Publish results in academic journals or conferences
- **Open Source**: Release as open-source project for community contribution
- **Benchmarking**: Create benchmark dataset for comparing recommendation approaches
- **Theoretical Analysis**: Formal analysis of algorithm convergence and optimality

---

9. Technical Documentation

9.1 System Architecture

Component Diagram:



Module Descriptions:

Data Module (`src/data/`):

- **loader.py**: Loads Excel files, extracts team/practice/month data
- **processor.py**: Normalizes data (0-3 → 0-1), builds team histories
- **validator.py**: Validates data quality, checks for missing values
- **practice_definitions.py**: Loads practice level definitions from Excel

ML Module (`src/ml/`):

- **similarity.py**: Cosine similarity calculations, finds K similar teams

- **sequences.py**: Practice transition learning, transition-matrix construction, popularity counts
- **policy.py**: Global monthly policy grid, cohort building, and blend scoring (`PolicyEngine`)
- **recommender.py**: Thin wrapper delegating to `PolicyEngine`

Validation Module (`src/validation/`):

- **backtest.py**: Rolling window backtest of the blend and primary/sensitivity aggregation
- **metrics.py**: Accuracy calculations, random baseline computation

API Module (`src/api/`):

- **routes.py**: FastAPI route definitions
- **service.py**: Service layer wrapping ML components
- **models.py**: Pydantic models for request/response validation
- **main.py**: FastAPI application setup

Dependencies:

- Data Module → ML Module (provides processed data)
- ML Module → Validation Module (provides recommender engine)
- API Module → ML Module, Validation Module, Data Module (orchestrates all)

9.2 Algorithm Details

Cosine Similarity Calculation:

Mathematical formulation:

```
similarity(A, B) = (A · B) / (||A|| × ||B||)
```

Where:

- A, B are practice maturity vectors (numpy arrays)
- A · B is dot product: Σ(A[i] × B[i])
- ||A|| is L2 norm: √(Σ(A[i]²))

Implementation:

```
from sklearn.metrics.pairwise import cosine_similarity
similarity = cosine_similarity(target_vector, team_vector)[0][0]
```

Practice Transition Matrix:

See §3.4 for the full construction algorithm over chronological improvement-bearing steps, with no edge asserted between same-step co-improvements. Direct code-level reference:

```
# per team: chronological list of practice-sets, one per improvement-bearing step
# (steps with zero improvements are skipped)
for prev_set, next_set in zip(improved_sets, improved_sets[1:]):
    for prev_practice in prev_set:
        for next_practice in next_set:
            transition_matrix[prev_practice][next_practice] += 1
```

Transition probability:

```
P(B | A improved) = count(A → B) / Σ_X count(A → X)
```

Recommendation Scoring Formula:

See §3.5 for the full hybrid scoring algorithm (normalize similarity and sequence scores separately, combine with `similarity_weight`, normalize again, filter maxed-out practices, rank). The final filter-and-rank step, as implemented:

```
recommendations.sort(key=lambda x: (-x[1], x[0]))  # deterministic tie-break by prac-
tice name
recommendations = recommendations[:top_n]
```

Normalization Procedures:

1. **Input Normalization** (0-3 → 0-1):

   ```
   normalized = raw_score / 3.0
   ```

2. **Score Normalization** (for combining):

   ```
   normalized = score / max_score
   ```

3. **Final Score Normalization** (for display):

   ```
   normalized = score / max_final_score
   ```

9.3 API Documentation

Base URL: `http://localhost:8000`

Endpoints:

1. GET /api/teams

- **Description**: Get all teams with metadata
- **Response**: List of team info objects

- **Example Response**:

```
[
  {
    "name": "AADS",
    "num_months": 10,
    "months": [200101, 200102, ...],
    "first_month": 200101,
    "last_month": 200110
  }
]
```

2. GET /api/teams/with-improvements

- **Description**: Get all teams and months where at least one practice improved
- **Response**: List of team-month pairs with improvement data

3. GET /api/teams/{team_name}/months

- **Description**: Get valid prediction months that the team has recorded, with a usable earlier baseline snapshot
- **Parameters**: `team_name` (path parameter)
- **Response**: Object with team name and months list
- **Example Response**:

```
{
  "team": "AADS",
  "months": [200105, 200106, 200107, ...]
}
```

4. POST /api/recommendations

- **Description**: Get recommendations for a team, using that prediction month's globally selected policy. The team must have both a recorded snapshot in the requested prediction month and a usable earlier baseline. `top_n` is pinned to 2 - any other value is rejected with a validation error rather than silently honored, and there is no `k_similar` (peer count is chosen by the policy, not the caller). If fewer than two non-maxed candidates remain, the response contains an empty `recommendations` list and an explanatory `message`
- **Request Body**:

```
{
  "team": "AADS",
  "month": 200105,
  "top_n": 2
}
```

- **Response**: Recommendation response with practices, explanations, and the selected policy's audit record

- **Example Response**:


```
{
  "team": "AADS",
  "month": 200105,
  "recommendations": [
    {"practice": "CI/CD", "score": 0.85, "current_level": 0.33, "why": "..."},
    {"practice": "Test automation", "score": 0.72, "current_level": 0.00, "why":
"..."}
  ],
  "validation": {...},
  "selected_policy": {
    "is_bootstrap": false,
    "peer_count": 10,
    "min_similarity": 0.75,
    "similarity_weight": 0.25,
    "sequence_weight": 0.25,
    "popularity_weight": 0.5,
    "popularity_recency_weight": 0.0,
    "completed_prior_months": [20200503],
    "mean_prior_hit_rate": 0.5714285714285714
  },
  "no_similar_teams_found": false,
  "message": null
}
```


4. POST /api/backtest

- **Description**: Run the backtest of the global monthly adaptive blend. No request body - there are no user-adjustable model parameters
- **Response**: `{ per_month_results, primary, sensitivity }` - `primary` covers prediction months with a complete 3-snapshot outcome window, `sensitivity` covers every prediction month; the two are never mixed

5. GET /api/stats

- **Description**: Get system statistics
- **Response**: System statistics including teams, practices, months, practice definitions

6. GET /api/sequences

- **Description**: Get learned improvement sequences
- **Response**: List of sequence transitions with probabilities

7. GET /api/example-data

- **Description**: Serve the raw Excel dataset file for in-browser preview (Statistics tab modal)
- **Response**: Excel file download (`combined_dataset.xlsx`)

8. GET /api/docs

- **Description**: Serve project documentation content as markdown
- **Response**: Raw markdown string rendered by the About modal in the frontend

There is no static all-history parameter optimizer and no `/api/optimize*` family of endpoints - see §6.5.

Error Handling:

- **400 Bad Request**: Invalid request parameters
- **404 Not Found**: Resource not found (e.g., team not found)
- **500 Internal Server Error**: Server error with error message

9.4 Data Format Specification

Excel File Structure:

Required columns:

- **Team Name** (column 1): Text identifier for team (e.g., "AADS", "Strikers")
- **Month** (column 2): Time period in YYMMDD format (e.g., 200101 = January 2020)
- **Practice Columns** (columns 3+): Practice names with maturity scores (0-3)

Example:

```
Team Name | Month   | CI/CD | TDD | DoD | Code Review | ...
AADS      | 200101  | 1     | 0   | 3   | 2           | ...
AADS      | 200102  | 2     | 0   | 3   | 2           | ...
Strikers  | 200101  | 3     | 2   | 3   | 3           | ...
```

Data Validation Rules:

- Team Name: Non-empty string
- Month: Numeric integer in the project's YYMMDD-style encoding (for example, `200101`)
- Practice scores: Integer in range [0, 3]
- Missing values: Filled with 0, normalized to 0.0

Processing Pipeline:

1. Load Excel file → DataFrame
2. Validate columns and data types
3. Fill missing values with 0
4. Normalize scores: score / 3.0
5. Build team histories: {team: {month: [practice_scores]}}

9.5 Configuration Parameters

There are no user-adjustable or per-request model parameters. `top_n` is pinned to 2. Every other knob below is chosen automatically by the global monthly policy selection (§6.5) - never by the caller, and never fixed at a single default:

Peer count - one of 5, 10, or 19 similar teams, chosen per prediction month **Minimum similarity threshold** - one of 0.0, 0.5, or 0.75, chosen per prediction month **Similarity / sequence / popularity weights** - one of 15 combinations of 0%/25%/50%/75%/100% summing to 100%, chosen per prediction month **Popularity recency weight** - one of 0%, 25%, 50%, 75%, or 100%, chosen per prediction month **Similarity look-ahead window and sequence recency window** - both fixed at exactly 2 observed snapshots; never part of the grid, never tunable

The selected policy for a given prediction month is reported in the `selected_policy` field of both the recommendations response and the backtest's per-month results (§9.3), so the actual values in effect are always visible even though they cannot be configured directly.

---

10. User Manual

10.1 System Overview

The Agile Practice Recommendation System is a web-based application that identifies likely next agile practices for teams based on organizational history. The system analyzes patterns from similar teams and improvement sequences to provide personalized recommendations.

Key Features:

- **Personalized Recommendations**: Get exactly two practice recommendations for an eligible team-month using that month's globally selected policy; if fewer than two practices remain, the interface explains why no list can be returned
- **Validation**: Run backtest validation to see how often recommendations align with later improvements, split into primary and sensitivity results
- **Statistics**: View system statistics and practice definitions
- **Sequences**: Explore learned improvement patterns

10.2 Installation Guide

See **docs/INSTALLATION.md** for detailed installation instructions.

Quick Installation:

1. Install Python 3.10+
2. Install dependencies: `pip install -r requirements.txt`
3. Start web server: `python src/web_main.py data/raw/combined_dataset.xlsx`
4. Open browser: `http://localhost:8000`

10.3 Getting Started

See **docs/QUICK_START.md** for a 3-step quick start guide.

Quick Start:

1. Install dependencies
2. Start web interface
3. Open http://localhost:8000 in browser

10.4 Using the Web Interface

Main Tabs (in order from left):

1. Statistics Tab (default):

- View system statistics (teams, practices, months)

- See practice definitions and maturity level descriptions
- Explore practice improvement frequencies

2. Backtest Validation Tab:

- No configuration form - there is nothing to adjust, since the monthly policy is the sole configuration authority
- Click "Run Backtest Validation" to validate on historical data
- View primary and sensitivity accuracy metrics, improvement factors, and the time-aware-popularity comparison, plus a per-month table showing each month's selected policy

3. Sequences Tab:

- View learned improvement sequences
- See transition probabilities between practices
- Expand sequences to see detailed transitions

4. Recommendations Tab:

- Select a team from the dropdown
- Select the displayed current-month → prediction-month pair
- Click "Get Recommendations"
- View top recommended practices with scores and explanations
- See validation summary if available

About/Documentation Modal:

- Click the "About" button in the header to open the documentation modal
- Renders the full project documentation in-browser using the `/api/docs` endpoint
- Allows reading methodology, algorithm details, and user manual without leaving the app

10.5 Using the CLI Interface

Starting CLI:

```
python src/main.py data/raw/combined_dataset.xlsx
```

Interactive Menu:

1. **Get Recommendations**: Enter team name and month
2. **Run Backtest**: Validate on historical data
3. **View Statistics**: See system statistics
4. **View Sequences**: See improvement sequences
5. **Exit**: Quit the application

Example Usage:

```
Select option: 1
Enter team name: AADS
Enter month (YYMMDD-style integer): 200105
[Shows recommendations]
```

10.6 Understanding Results

Recommendations:

- **Practice Name**: The recommended agile practice
- **Score**: Recommendation score (0-1, higher is better)
- **Current Level**: Team's current maturity level (0-1)
- **Why**: Explanation of why this practice was recommended

Validation Summary:

- **Practices Improved**: Number of practices that actually improved
- **Accuracy**: Percentage of recommendations that matched actual improvements
- **Validation Window**: Subsequent observed-improvement window (the target month and the following two recorded months)

Backtest Results:

- **Overall Accuracy**: Percentage of validated recommendations
- **Random Baseline**: Expected accuracy with random selection
- **Improvement Factor**: How many times better than random
- **Per-Month Results**: Accuracy broken down by month
- **Supplementary Rank-Aware Metrics**: Precision@N, Recall@N, and MRR, each shown against its own random baseline — stricter, order-sensitive alternatives to the headline Accuracy (see §3.6 and §6.2)

Sequence Patterns:

- **From Practice → To Practice**: Shows which practices typically follow others
- **Count**: Number of times this transition occurred
- **Probability**: Likelihood of this transition (0-1)

10.7 Troubleshooting

Server won't start:

- Check that port 8000 is not in use
- Verify data file exists: `ls data/raw/combined_dataset.xlsx`

- Make sure all dependencies are installed: `pip list`

Can't access http://localhost:8000:

- Make sure the server started successfully
- Check firewall settings
- Try http://127.0.0.1:8000 instead

Import errors:

- Activate virtual environment if using one
- Reinstall dependencies: `pip install -r requirements.txt`

No recommendations shown:

- Check that team has data for selected month
- Choose a valid global prediction month and a team with a baseline snapshot before it
- Check that team has improvements in validation window

Backtest takes too long:

- Normal: the first backtest run after startup sweeps all 675 candidate policies per prediction month and can take up to a couple of minutes; subsequent runs reuse `PolicyEngine`'s caches and are much faster
- The interface shows estimated progress while the backtest runs
- Check server logs for progress

---

11. Code Documentation

11.1 Project Structure

Directory Tree:

```
agile-prediction-mvp/
├── src/
│   ├── data/
│   │   ├── __init__.py
│   │   ├── loader.py          # Excel file loading
│   │   ├── processor.py       # Data normalization and processing
│   │   ├── validator.py       # Data validation
│   │   └── practice_definitions.py  # Practice definitions loader
│   ├── ml/
│   │   ├── __init__.py
│   │   ├── similarity.py      # Cosine similarity engine
│   │   ├── sequences.py        # Practice Transition Model + popularity counts
│   │   ├── policy.py           # Global monthly policy grid, cohorts, blend scoring
│   │   └── recommender.py     # Thin wrapper delegating to PolicyEngine
│   ├── validation/
│   │   ├── __init__.py
│   │   ├── backtest.py         # Backtest validation of the blend
│   │   └── metrics.py          # Accuracy metrics
│   ├── api/
│   │   ├── __init__.py
│   │   ├── main.py             # FastAPI application
│   │   ├── routes.py           # API route definitions
│   │   ├── service.py          # API service layer
│   │   └── models.py           # Pydantic models
│   ├── interface/
│   │   ├── __init__.py
│   │   ├── cli.py              # Command-line interface
│   │   └── formatter.py        # Output formatting
│   ├── main.py                 # CLI entry point
│   └── web_main.py             # Web server entry point
├── web/
│   ├── index.html              # Web interface
│   ├── static/
│   │   ├── css/
│   │   │   └── style.css       # Styles
│   │   ├── js/
│   │   │   ├── app.js          # Frontend logic
│   │   │   └── api.js          # API client
│   │   └── favicon.svg          # Favicon
├── data/
│   └── raw/
│       └── combined_dataset.xlsx  # Input data file
├── tests/                       # Unit, integration, and browser tests
├── docs/
│   ├── PROJECT_DOCUMENTATION.md # This file
│   ├── INSTALLATION.md          # Installation guide
│   └── QUICK_START.md           # Quick start guide
├── requirements.txt             # Python dependencies
└── README.md                    # Project overview
```

11.2 Module Descriptions

Data Module (`src/data/`):

loader.py - DataLoader class:

- `load()`: Loads Excel file into pandas DataFrame
- Identifies practice columns automatically
- Extracts teams, practices, and months

processor.py - DataProcessor class:

- `process()`: Normalizes scores (0-3 → 0-1), builds team histories
- `get_team_history(team_name)`: Returns team's history as {month: [scores]}
- `get_all_teams()`: Returns list of all teams
- `get_all_months()`: Returns list of all months

validator.py - DataValidator class:

- `validate()`: Validates data quality, checks for missing values
- Reports data quality issues
- Handles edge cases

practice_definitions.py - PracticeDefinitionsLoader class:

- `get_definitions()`: Loads practice level definitions from Excel
- `get_remarks()`: Loads practice remarks
- Gracefully handles missing file

ML Module (`src/ml/`):

similarity.py - SimilarityEngine class:

- `find_similar_teams(team, month, k)`: Finds K most similar teams
- Uses cosine similarity from scikit-learn

sequences.py - SequenceMapper class:

- `learn_sequences()`: Learns transition matrix and popularity counts from all historical data
- `learn_sequences_up_to_month(max_month)`: Learns up to specific month (for backtesting/policy scoring)
- `get_typical_next_practices(practice, top_n)`: Returns practices that typically follow
- `get_practice_popularity()`: Returns organization-wide improvement counts, most-improved first
- Uses caching to avoid recomputation

policy.py - PolicyEngine class:

- `recommend(team, prediction_month)`: Selects the month's policy and returns exactly two scored candidates when eligible, or an empty result marked `insufficient_practices`
- `select_policy(prediction_month)` / `select_popularity_arm(prediction_month)`: Global monthly policy selection over the 675-combination grid
- `evaluable_cases(prediction_month)`: Builds the fixed backtest cohort, independent of any policy
- `explain_practice(team, prediction_month, practice)`: Explains why a practice was (or would be) recommended

recommender.py - RecommendationEngine class:

- Thin wrapper constructing and delegating to a `PolicyEngine`; kept so existing constructor call sites don't need to change
- `recommend(team, prediction_month)` / `get_recommendation_explanation(...)`: delegate directly

Validation Module (`src/validation/`):

backtest.py - BacktestEngine class:

- `run_backtest()`: Runs the rolling window backtest of the blend - no config parameter, no user-adjustable model parameters
- Validates recommendations against actual improvements, split into primary and sensitivity aggregates

API Module (`src/api/`):

routes.py - API route definitions:

- `create_routes(service)`: Creates FastAPI router with all endpoints
- Defines GET/POST endpoints for all API operations
- Handles errors and exceptions

service.py - APIService class:

- Wraps ML components for API use
- `get_recommendations(...)`: Gets recommendations with validation
- `run_backtest(...)`: Runs backtest validation
- `get_system_stats()`: Returns system statistics

models.py - Pydantic models:

- Request and response models, including `RecommendationRequest`, `RecommendationResponse`, and `BacktestResponse`
- Ensures type safety and validation

11.3 Key Classes and Functions

Main Classes:

PolicyEngine (`src/ml/policy.py`):

- **Purpose**: Owns the global monthly policy grid, cohort building, and blend scoring
- **Key Methods**:
  - `recommend()`: Main recommendation generation, using the prediction month's selected policy
  - `select_policy()` / `select_popularity_arm()`: Global monthly policy selection
  - `evaluable_cases()`: Fixed backtest cohort, independent of any policy
  - `explain_practice()`: Provides explanations
- **Dependencies**: SimilarityEngine, SequenceMapper, DataProcessor

RecommendationEngine (`src/ml/recommender.py`):

- **Purpose**: Thin wrapper delegating to `PolicyEngine`, kept for a stable constructor shape
- **Key Methods**:
  - `recommend()`, `get_recommendation_explanation()`: delegate directly to `PolicyEngine`
- **Dependencies**: `PolicyEngine`

SimilarityEngine (`src/ml/similarity.py`):

- **Purpose**: Finds similar teams using cosine similarity
- **Key Methods**:
  - `find_similar_teams()`: Finds K most similar teams
- **Dependencies**: DataProcessor

SequenceMapper (`src/ml/sequences.py`):

- **Purpose**: Learns empirical practice transition patterns and organization-wide popularity counts
- **Key Methods**:
  - `learn_sequences()`: Learns from all data
  - `learn_sequences_up_to_month()`: Learns up to specific month
  - `get_typical_next_practices()`: Returns next practices
  - `get_practice_popularity()`: Returns organization-wide improvement counts
- **Dependencies**: DataProcessor

BacktestEngine (`src/validation/backtest.py`):

- **Purpose**: Validates the blend using rolling window backtesting, split into primary and sensitivity aggregates
- **Key Methods**:

- `run_backtest()`: Runs rolling window backtest
- `_aggregate_scope()`: Aggregates one scope (primary or sensitivity)

- **Dependencies**: RecommendationEngine (via its `PolicyEngine`), DataProcessor

Important Algorithms: see §9.2 (Algorithm Details) for direct code-level snippets of cosine similarity, transition matrix construction, and hybrid scoring, and §3.3-3.5 for the full algorithm design each implements.

11.4 Code Examples

Getting Recommendations:

```
from src.ml import RecommendationEngine, SimilarityEngine, SequenceMapper
from src.data import DataProcessor, DataLoader

# Load and process data
loader = DataLoader("data/raw/combined_dataset.xlsx")
df = loader.load()
processor = DataProcessor(df, loader.practices)
processor.process()

# Initialize ML components
similarity_engine = SimilarityEngine(processor)
sequence_mapper = SequenceMapper(processor, loader.practices)
recommender = RecommendationEngine(similarity_engine, sequence_mapper, loader.prac-
tices)

# Get recommendations - exactly two when eligible, using that month's selected policy
result = recommender.recommend("AADS", 200105)
if result.insufficient_practices:
    print("Fewer than two practices remain to improve")
for practice in result.practices:
    print(f"{practice}: {result.scores[practice]:.2f} (current: {re-
sult.current_levels[practice]:.2f})")
print(f"Selected policy: {result.selected_policy}")
```

Running Backtest:

```
from src.validation import BacktestEngine

# No config - there are no user-adjustable model parameters
backtest_engine = BacktestEngine(recommender, processor)
results = backtest_engine.run_backtest()

print(f"Primary accuracy: {results['primary']['overall_accuracy']:.1%}")
print(f"Primary improvement factor: {results['primary']['improvement_factor']:.1f}x")
print(f"Sensitivity accuracy: {results['sensitivity']['overall_accuracy']:.1%}")
```

Using API:

```
from src.api.service import APIService
from src.api.routes import create_routes
```

```
from fastapi import FastAPI

# Initialize service
service = APIService(recommender, processor)

# Create FastAPI app
app = FastAPI()
app.include_router(create_routes(service))

# Run server
import uvicorn
uvicorn.run(app, host="0.0.0.0", port=8000)
```

Appendix A: File Locations

Source Code:

- Main code: `src/` directory
- Web interface: `web/` directory
- Tests: `tests/` directory

Data Files:

- Input data: `data/raw/combined_dataset.xlsx`
- Practice definitions: `data/raw/practice_level_definitions.xlsx`

Documentation:

- This file: `docs/PROJECT_DOCUMENTATION.md`
- Quick start: `docs/QUICK_START.md`
- Installation: `docs/INSTALLATION.md`
- Overview: `README.md`

Configuration:

- Dependencies: `requirements.txt`
- Startup scripts: `start_mac_linux.sh`, `start_windows.bat`

End of Documentation